CareCompanion: A Caregiver-Mediated Android System for Older Adults, with Fail-Honest Alerting

Suyash S. Humne[1], Devanshi Pandey[2], Deon Menezes[1], Sonali Wankhede[1], and Balwant Kumar Singh[2]

[1] Department of Artificial Intelligence and Data Science, K. J. Somaiya School of Engineering, Somaiya Vidyavihar University, Mumbai, India
{suyash.humne,deon.m,sonali.w}@somaiya.edu

[2] Department of Electronics and Computer Engineering, K. J. Somaiya School of Engineering, Somaiya Vidyavihar University, Mumbai, India
{devanshi.pandey,balwantkumar}@somaiya.edu

Abstract. Mobile applications that help older adults manage medication and summon help are typically evaluated on the legibility of the older adult's interface. We argue a second property matters at least as much and is rarely examined: whether the system tells the truth when it fails. We present CareCompanion, an implemented Android system for caregiver-mediated elder care built on three commitments. An asymmetric dual-surface architecture places all configuration on the caregiver's device, leaving the older adult a zero-configuration surface of seven destinations; measured from source, that surface has a median declared text size of 20 sp against 13 sp on the caregiver surface, and every primary control equals or exceeds 64 dp against a 48 dp platform minimum. Out-of-band device binding resolves an identity gap federated sign-in creates: a single-use numeric code binds the older adult's device to a care profile, replacing phone-number matching. Fail-honest alerting requires the interface never to assert an outcome it has not observed; we name the event each confirmation reflects, and show our own first implementation violated the rule. As a single-system case study, an audit found 13 defects, of which 8 would have failed with no user-visible signal, including two features written, stored and never delivered; a blind re-rating supported that figure while showing it turns on three borderline judgements. We report the taxonomy, the mechanisms addressing each class, and what remains unevaluated: no user study, and no rate limiting on the binding code.

Keywords: Elder care · Mobile health · Caregiver-mediated systems · Accessibility · Dependability · Android · Case study.

1 Introduction

India is experiencing rapid population ageing: the United Nations Population Fund projects its population aged 60 and over will grow by 134% between 2022

and 2050, and that aged 80 and over by 279% [1]. Digital access remains uneven, since internet use declines substantially with age [2], and among older adults in rural India, particularly women, shared handsets and limited digital literacy are the norm [3,4]. Randomised evidence suggests mobile reminder tools help with medication non-adherence but do not solve it [5,6].

The usability literature is clear about what makes an interface hard for an older adult: small text is repeatedly the most-cited barrier, touch targets are too small for reduced dexterity, icons are misread, and navigation depth exceeds working memory [7,8,9]. What it largely assumes is that the older adult configures the system. In practice care is dyadic; Quinn et al. found most interaction occurred between caregiver and older adult rather than the older adult alone [12]. If the caregiver configures, the older adult's surface need not be a simplified management application. It can be a different application altogether.

A second assumption is more consequential and under-examined: that when the system reports success, it has succeeded. Emergency and reminder paths are unreliable in ways the application can observe but frequently does not check. Doze and App Standby defer alarms and network access during idle [14], manufacturer power management is more aggressive still, and push depends on connectivity an older adult at home may not have. An interface that renders a green tick on an emergency alert it never delivered is not merely unhelpful; in the one situation the system exists for, it is actively harmful.

Contributions. We contribute (i) an *asymmetric dual-surface architecture* in which configuration authority and interface complexity are deliberately unequal across two clients over one care record, measured from the implementation rather than asserted; (ii) *out-of-band device binding*, a single-use code repairing an identity gap federated sign-in introduces, with an explicit threat model; (iii) *fail-honest alerting*, under which the interface never asserts an unobserved outcome, for which we name the exact event each confirmation reflects and report that our own implementation violated the rule; and (iv) an *empirical defect taxonomy* presented throughout as a single-system case study. We report conformance, interaction cost and defect data measured from the artefact, have run no user study, and make no usability, adherence or clinical claim (Section 8).

2 Related Work

Systematic reviews of mobile design for older adults consistently identify text size, target size, icon interpretability and navigation complexity as the dominant barriers [7,8,9], and guideline syntheses converge on enlarged type and targets, shallow navigation and error tolerance [10,11]. Our elder surface implements this consensus; the contribution is not the guidelines but the demonstration that offloading configuration to a second surface is what makes conformance affordable. On adherence, Poorcheraghi et al. report a randomised trial finding improved adherence and reduced readmission [5], Salmensuu et al. review reminder technologies for home-dwelling older citizens [6], and Wiecek et al. report real-world

engagement evidence [15]. These evaluate outcomes; they do not generally report whether reminders were delivered by the operating system as scheduled, which Section 3.4 treats as a first-class engineering problem.

Quinn et al. observed interaction was predominantly caregiver-mediated [12], Benmessaoud et al. designed a dyadic module for shared care [13], and Nimmanterdwong et al. review human-centred design practice [16]. This work establishes the dyad as the unit of design; we extend it architecturally, building two surfaces of deliberately unequal complexity over one access-controlled record rather than one application used by two people. Personal emergency response systems have a documented failure mode. Hamill et al. report estimates attributing as much as 85% of call-centre traffic to false alarms [17], which motivates our cancellable grace period. The complementary failure, an alarm believed raised that reached nobody, is less discussed. Across these strands, legibility and efficacy are well covered; delivery honesty and deployability under platform policy are not.

3 System Design

3.1 Asymmetric Dual-Surface Architecture

The system presents two Android surfaces backed by one record. To avoid ambiguity we fix a counting rule and use it throughout: a *navigation destination* is one entry registered in the client's navigation graph, counted from the source. By that rule the caregiver surface has 20 destinations and the elder surface has 7, plus a one-time binding screen reached only at set-up and not part of the running surface.

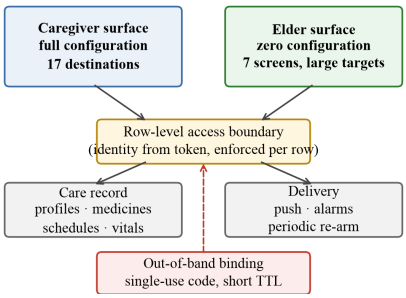


Fig. 1. System structure. Two clients of deliberately unequal complexity share one care record behind a per-row access boundary that derives identity from the caller's token. The out-of-band binding step (bottom) establishes which physical device belongs to which older adult, a link federated sign-in does not provide.

The caregiver surface carries the full management vocabulary: profiles, medicines, schedules, reminders, contacts, vitals, adherence history, shortcuts, shared access and an emergency monitor. The elder surface carries a home screen, an alarm

control, today's medicines, contacts, vitals entry, media shortcuts and a settings screen of three controls. Contacts there are photo-first, so recognition does not depend on reading a name.

The asymmetry is the design. Because no configuration occurs on the elder surface, it has no forms to validate, no destructive actions to guard, no navigation deeper than two levels and no state the older adult can corrupt, which is what makes generous type and target sizing affordable, since a screen accommodating a scheduling form cannot also give every control 96 dp of height. Both surfaces authenticate with a federated identity provider and address the same record through a per-row access boundary: thirty-three row-level policies across seventeen tables derive the caller's identity from the verified token rather than any client-supplied parameter, so a modified client cannot widen its own visibility.

3.2 Out-of-Band Device Binding

Elder-care systems must answer a question ordinary applications do not: which physical device belongs to the person being cared for? The caregiver creates the profile, so the profile cannot identify its subject by who created it.

Our first implementation matched on phone number, which works under telephone-based authentication and fails completely under federated sign-in, which returns an email address and a provider subject identifier and no telephone number. After migrating, the stored telephone number for every account was the empty string and the matching predicate could never be satisfied. The failure was silent: links sat in a pending state while the resolution routine compared an empty string against a real number indefinitely. Section 5.3 returns to this as an instance of a general hazard.

The repair is an out-of-band binding primitive. The caregiver requests a six-digit code bound to the profile; it is single-use and expires after seven days. On the older adult's device, after federated sign-in, entering the code claims the profile. The channel by which the code travels is deliberately unspecified: spoken aloud, sent over any messaging application, or typed by the caregiver on the older adult's device. The reason is that the caregiver is present in the overwhelming majority of set-ups [12].

Threat model, and what review changed. We state what the primitive guarantees, because an earlier draft described the code as "short-lived", which a seven-day lifetime does not justify, and claimed security properties the implementation did not have.

What held already. Redemption is authenticated: the endpoint rejects any caller without a verified federated token and a provisioned account. We verified this against the deployed service, which refuses guessed codes from an unauthenticated client before examining them. The code is single-use, a bound profile cannot be silently rebound, and unknown and expired codes return an identical message.

What review exposed, and what we then fixed. Inspection of all fifteen database migrations found no attempt limit, no lockout and no rate limiting anywhere, against a code space of 10^6 and a seven-day window; and redemption was not atomic, since both routines read the code row and updated it later without locking, so two concurrent redemptions could each observe it as unredeemed. We have since added a per-account failure counter that refuses further attempts after five failures in fifteen minutes and is cleared by a successful redemption, and taken a row lock on the code for the remainder of the transaction so a concurrent redemption blocks and then observes the row as spent.

What remains. An already-redeemed code still returns a message distinct from the unknown-code message, which is in principle an enumeration oracle. We retain it deliberately: collapsing it would tell an older adult holding a spent code only that the code is invalid, removing the hint that lets them recover without assistance, and attempt limiting makes enumerating 10^6 through that oracle infeasible. The seven-day lifetime and the 10^6 space are unchanged. The rate limiter has been deployed and the unauthenticated path tested, but the limiter itself has not been exercised end-to-end under an authenticated session, so we claim it as implemented and not as measured.

3.3 Fail-Honest Alerting

We adopt a single rule: the interface never asserts an outcome it has not observed. Applied to the emergency path, this produces the machine in Fig. **??**.

Pressing the alarm control begins a five-second cancellable grace period. This is a direct response to the dominant failure mode of personal emergency response systems — Hamill et al. report estimates of up to 85% of call-centre traffic arising from accidental activation [17] — and it costs a genuine emergency five seconds.

What each confirmation actually observes. A reviewer rightly pressed us here, and the honest answer is narrower than "the alert was delivered" suggests. The client makes up to three attempts, with increasing delay, to write an alert row to the server; that row is what the fan-out and push machinery consume. The confirmation shown to the older adult is permitted by exactly one event: **the server acknowledged persistence of the alert record**. It is not push acceptance by the provider, not delivery to a caregiver device, not display of a notification, and not acknowledgement by a human caregiver. The failure state is entered when that write was unacknowledged after the final attempt; it then states plainly that the alarm was not delivered and offers two large controls, one handing the device's messaging application a pre-composed message with a location link, the other dialling the primary contact.

The distinction is not pedantic. Caregiver acknowledgement exists in the data model and appears on the caregiver's monitor, but is not propagated back to the older adult's device. Closing that loop, so the elder surface can distinguish "your family's phone received this" from "your family has seen this", is the most valuable extension of the discipline and is planned work. We note too that the

shipped build's string reads "Sent to your family's phones", which overstates what was observed by the rule above; it is corrected in the current source to name server receipt.

An earlier revision of this screen displayed a success confirmation unconditionally. It was possible for an older adult to be shown a green tick when the emergency event had not reached the server at all. We regard this class of defect as the most serious a care system can contain, because it converts a recoverable failure into an unrecoverable one: a user who believes help is coming stops seeking it. That our own implementation of the rule still contained a residual overstatement, found only under review, is itself evidence for the paper's thesis.

3.4 Redundant Reminder Arming

Reminders must fire on a device the application may not be running on, under an operating system suppressing background work [14]. An early revision armed the day's alarms only when the older adult opened the application, so an older adult who did not open it received no reminders that day, precisely the user for whom they exist. We now arm through three independent paths: on application open; from a periodic worker re-arming the current day every four hours, skipping answered doses; and from a boot-completed receiver re-arming from a persisted set, since alarms do not survive a restart. Two correctness properties proved necessary. Alarm keys must be namespaced by source, because the key's hash serves as both pending-intent request code and notification identifier, so an un-namespaced key let a general reminder and a medication dose replace one another. And answering a dose must cancel its pending alarm, or an older adult who takes a tablet early is told to take it again at the scheduled time: a prompt toward double-dosing generated by an adherence tool.

3.5 Permission-Minimal Degradation and Localisation

Emergency features in the literature routinely assume the application may send a message directly. Google Play restricts messaging and call-log permissions to applications whose primary function is messaging or telephony [18,19], so we removed direct sending and dialling entirely: the application hands a pre-composed message to the device's own messaging application through a standard intent and the user presses send. This costs one tap in the degraded path and removes two restricted permissions, reducing the declared set to eleven, none restricted. The elder surface is localised into English, Hindi, Marathi and Gujarati and offers three type scales and a high-contrast mode composing with the operating system's own scaling, since English-only interfaces are a documented barrier for older adults in India [3,4].

4 Implementation

Both clients are native Android applications in Kotlin using Jetpack Compose, targeting API level 36 with a minimum of API 24. Persistence and access con-

trol use Supabase-hosted PostgreSQL with row-level security over its generated REST interface; privileged operations are database routines with definer rights. Push uses Firebase Cloud Messaging with data-only messages, since notification-payload messages bypass the application's handler when backgrounded and would prevent the client applying its own delivery rules.

The suite comprises 142 declared test methods across 24 classes, which execute as 147 cases because one method is parameterised over six HTTP status codes. It runs on the JVM with a shadowed Android runtime, concentrated where silent failure is most likely: alarm scheduling and cancellation, offline durability, delivery-contract behaviour of the push path, translation-table integrity, and a regression test per defect in Section 5.3. It is a unit-level suite; device-level fault injection is reported separately in Section 5.4.

Preparing this revision surfaced an instance of our own thesis in our own tooling. The Gradle configuration sets `ignoreFailures` on unit tests, so the build prints BUILD SUCCESSFUL irrespective of results; an independent run found all tests passing in the debug variant but ten Compose interface tests failing in the release variant, a test-configuration defect invisible from the build's own exit status. We had read that exit status as evidence the suite was green. A build that reports success it has not observed is the same defect class as an interface that does, and we report it rather than quietly correcting it.

Table 1. Implementation size, measured from the repository. Counts are of *source files*, not of navigation destinations; the two should not be conflated.

Component	Files	Lines
Caregiver surface	27	3,942
Elder surface	6	1,271
Data access and models	12	1,082
Reminder and alarm engine	4	329
Notification and push	2	147
Authentication	1	79
Application total (Kotlin)	83	13,473
Automated tests (Kotlin)	26	3,010
Database migrations (SQL)	15	1,121

The test suite comprises 142 automated tests executing on the Java virtual machine with a shadowed Android runtime, requiring no device or emulator. Coverage is concentrated where silent failure is most likely: alarm scheduling and cancellation, offline durability of adherence records, delivery-contract behaviour of the push path, translation-table integrity across the four languages, and regression tests for each defect in Section 5.3. Static analysis reports no errors at release configuration. We note the limits of this suite in Section 5.3: it is a unit-level suite on a shadowed runtime, and it contains no device-level or network-level fault injection.

5 Evaluation

We evaluate three properties measurable from the artefact: conformance of the elder surface to age-friendly sizing guidance, interaction cost, and the defect profile. All figures are generated by a script parsing the application source directly, so they cannot drift from the implementation. None is a usability result.

5.1 Conformance of the Elder Surface

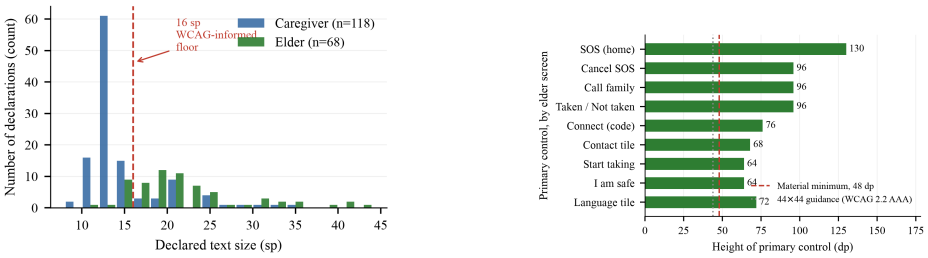


Fig. 2. Left: declared text sizes across the two surfaces, parsed from source; medians 13 sp (caregiver) and 20 sp (elder). Right: height of the primary control on each elder screen; median 76 dp, emergency control 130 dp.

The elder surface has a median declared text size of 20 sp against 13 sp for the caregiver surface, and 16.4% of its declarations fall below 16 sp against 79.7% on the caregiver surface; the residual small sizes are supporting captions rather than primary content. Because the elder surface composes a user-selected scale of 1.0, 1.15 or 1.3 with the operating system's own font scaling, the effective floor rises further when either is increased. The smallest primary control is 64 dp, one third above the 48 dp platform minimum [20] and above the 44×44 recommendation associated with WCAG 2.2 AAA [21], noting WCAG is specified in CSS pixels rather than density-independent pixels, so the comparison is indicative. We state a limit a reviewer correctly identified: these are *declared heights* of one control, a one-dimensional static property. They do not establish effective touch width, spacing between adjacent targets, or any observed user outcome.

5.2 Task Interaction Cost

Counted analytically from the implemented flows, raising an alarm costs one tap. Confirming a dose from its reminder costs three, after a change routing the notification directly to the medication screen rather than the home screen; previously it cost four. The saving sounds trivial and is not: each intermediate screen is an opportunity for an older adult to lose the thread, and an unconfirmed dose is recorded as missed and shown to the caregiver as possible non-adherence. Interaction cost here is coupled to data quality, not only convenience. These counts are analytic, not observed task times.

5.3 Defect Profile: A Single-System Case Study

This is a case study of one system audited by its own developers. It is not a population estimate. After the system was feature-complete and had been exercised on physical devices, we reviewed all 83 source files line by line together with the database schema and access policies, recording every defect and classifying each by whether a user could have detected it from the interface alone. Every defect was subsequently corrected, each correction traceable to a commit and to the regression test added for it.

Table 2. Defects found in the audit window, with the authors' classification of whether a user could have detected each from the interface alone.

Class	n	Detectable from the interface?
Unreachable feature	3	No
Lost state / race	3	No
Silent failure	2	No
Perception / contrast	2	Yes
Scheduling correctness	1	Yes
Interaction cost	1	Yes
Visual consistency	1	Yes

Thirteen defects were found. On the authors' classification, 8 of 13 were silent: the operation failed or never occurred while the interface reported success or reported nothing. Three classes deserve comment. **Unreachable features (3):** two complete features — general reminders for hydration and activity, and the caregiver invitation path — were written, exercised by the caregiver, stored correctly, and delivered to nobody, while the caregiver's interface behaved as though they worked. **Lost state (3):** three view models rebuilt their state object when a background refresh completed, discarding the completion flag the screen used to navigate, so a save that had already succeeded appeared to do nothing; it reproduces only when the refresh lands after the save, which is why it survived manual testing. **Null failure messages (1 class, 17 sites):** every repository write ends by taking the first element of the server's response, and when access policy hides the returned row that list is empty and the exception carries no message, which seventeen call sites rendered as an empty error.

Because the classification was made by the system's developers, we obtained a blind re-rating: each defect was reduced to a mechanism-only description with the label removed and classified against a written rubric by a rater with no access to the repository, the paper, or the original labels. We report this precisely. The rater was a large language model, not an independent human expert, and does not substitute for one. On the twelve items comparable between the sheets the raters agreed on eleven ($\kappa = 0.82$), differing only on whether an empty error message is a visible failure signal; the rater called it visible, which would move the headline from 8 to 7 of 13.

The more useful result is a sensitivity the single-rater count concealed. The rater gave its lowest confidence (2 of 5) to the three lost-state defects, reasoning that a form which does not navigate away after a successful save is arguably a visible symptom. Were those reclassified, the silent proportion would fall from 8/13 to 5/13. The headline therefore rests on one judgement applied to three items, and we state it that way rather than reporting 62% as though it were robust. We also note what no re-rating can repair: the defects were *selected* by the developers, and no rating exercise recovers defects never recorded.

Two observations are candidates for transfer, offered as hypotheses rather than findings. Identity-model migration produces latent dead features: moving to federated sign-in silently invalidated every path using a telephone number as a join key, and nothing failed loudly. And unit suites do not detect unreachable features: our 142 tests passed throughout the period two features delivered nothing, because each component behaved correctly in isolation and the defect lay in the absence of a caller.

5.4 What Fault Injection Found

The audit above read the source. To test the platform's behaviour rather than the code's, we wrote instrumented tests running against the real `AlarmManager` on an emulator. We state the extent of this plainly, because it is partial: one run of six tests has been executed, on an emulator rather than a handset; the revised eight-test suite and the scripted injection of Doze, reboot, permission denial and connectivity loss have been written but *not run*, so this paper reports no result for any of them. The tests assert that an alarm was *accepted* by the platform, not that it fired.

That single partial run was still enough to find a defect the line-by-line audit had missed, and it is the kind the audit was least able to see. Arming replaces a persisted set of the day's doses. It did not cancel alarms for keys the new set dropped, so a dose removed from the schedule stayed armed with the platform while nothing tracked it any longer: the sign-out path walks the persisted set and so could not reach it, and it still rang. Both production arming paths — the older adult opening the application, and a four-hourly worker — pass the current list, so a caregiver deleting or re-timing a medicine during the day left the older adult being told to take it at the old time. An adherence tool generating a double-dose prompt is the failure this system exists to prevent, and it was silent until the moment it rang.

Two methodological points follow. First, the defect is invisible to source review of any one function: each of arming, cancelling and persisting is correct alone, and the fault is in the state they jointly leave behind, the same shape as the unreachable features of Section 5.3, where the defect lay in the absence of a caller. Second, our first version of these tests reported two failures against correct behaviour and one pass that asserted nothing, because the oracle tested whether a pending-intent token existed rather than whether an alarm was live. We record this because a test that reports a result it has not observed is the

same defect class as an interface that does, and we found it only because the run was cross-checked against the platform's own alarm table.

The repair then failed the same way, which is the most instructive part. Both arming paths obtain the day's reminders inside a construct that yields an empty list when the request fails, and our fix read that emptiness as authoritative, so a single timed-out request would have disarmed every general reminder for the day, silently. Review caught it before it shipped, and the same shape turned out to predate the fix on the adherence read, where a failed load makes an answered dose look pending and re-arms it. The rule this paper applies to interfaces — never assert an outcome you have not observed — has an exact analogue one layer down: never act on data you did not receive. An empty result and a failed request are not the same value, and a language construct that collapses them invites precisely the silent failures catalogued here.

5.5 Deployability

The application declares eleven permissions, none restricted; two restricted permissions in an earlier revision were removed in favour of the user-mediated path of Section 3.5. The release artefact is 20 MB. It has not been published to the store.

5.6 Threats to Validity

The defect study was classified by the system's own developers; we mitigated this with the blind re-rating of Section 5.3 rather than leaving it unchecked, but the rater was automated and an independent human re-rating remains outstanding. The audit window covers the period after feature completion, not the whole development history, so the count is not a defect density. Conformance measurements are static properties of the source and say nothing about perception or comprehension, and tap counts are analytic. Device-level fault injection is started but incomplete (Section 5.4): one partial run on an emulator, no handset, and no completed Doze, reboot, permission or connectivity measurement. An emulator runs a stock image, so even a complete run would say nothing about the manufacturer power-management layers that Section 3.4 identifies as the harder case. The dependability claims in this paper are therefore claims about *design discipline*, not about measured robustness.

6 Discussion

6.1 Silence Is the Dominant Risk

Most defects in this feature-complete care application were invisible from the interface, which matters more here than it would for, say, a photo application. The older adult often cannot diagnose a silent failure, and the caregiver — the person who could — is looking at a different device also reporting success. A

missing reminder produces no artefact at all: its absence is indistinguishable from a day with no medication due, and the adherence record the caregiver trusts will attribute the gap to the older adult. We therefore propose an explicit honesty discipline: acknowledge only observed outcomes and name in the interface which outcome was observed; give every failure path a message a lay reader can act on; and, for any feature whose value is delivery, test the delivery rather than the storage.

6.2 Asymmetry as an Accessibility Budget

The measured gap between the surfaces results not from applying guidelines harder on one screen but from removing work from it: type and target size compete with information density. By moving configuration to the caregiver, the elder surface gains an accessibility budget to spend on sizing and shallow navigation. We suggest this reframing, asymmetry as a budget rather than as simplification, is the transferable idea, and that it is a design insight rather than a new algorithm. Relatedly, store permission policy determined our emergency path's architecture, and a design that cannot be distributed cannot accumulate the longitudinal evidence the field needs.

7 Scalability and Path to Evaluation

Care circles are disjoint and queries are scoped to a profile, so growth should be horizontal; reminder scheduling happens on the device, so the server is not a timing authority. The identified risks are ordinary rather than architectural: push-token churn needs re-synchronisation on session restore, which we implement; the scheduled scan for unanswered doses is a single point of failure needing partitioning; and per-row policy evaluation grows with policy count. Large-scale performance has not been benchmarked.

The next step is measurement. The fault-injection suite of Section 5.4 must be completed on physical handsets across manufacturer power-management regimes, which the emulator cannot represent. Beyond that, a formative study with older adult–caregiver dyads (as a reviewer suggested) measuring the delivery-level outcomes the literature usually assumes: the proportion of scheduled reminders actually presented by the device, time from alarm activation to caregiver acknowledgement, and frequency of the degraded emergency path.

8 Limitations

We state these plainly because the paper's credibility depends on it. **No user study has been conducted**: nobody outside the development team has used the system under observation, and every conformance number is a static property of the implementation. **No field measurement of delivery**: we have no distribution of delivery latency across manufacturers and power-management

regimes. **No independent human rating**: the taxonomy comes from one application audited by its developers, checked only by the automated re-rating of Section 5.3. **Limited adversarial testing**: the binding primitive's rate limiter is implemented but not exercised end-to-end under an authenticated session. **Scope is ageing, not disability in general**: we have not evaluated the system with users whose impairments arise independently of ageing, and make no claim of general accessibility. **Localisation is untested with native speakers**. **Not a medical device**: the system gives no diagnostic or dosing advice.

9 Conclusion

CareCompanion rests on three commitments: an asymmetric dual-surface architecture that buys the older adult an accessibility budget by removing configuration from their device; out-of-band binding that repairs an identity gap federated sign-in introduces; and fail-honest alerting, under which the interface never asserts an outcome it has not observed. The asymmetry is measurable: a median 20 sp against 13 sp, every primary control at or above 64 dp. The case study shows why the honesty discipline matters: on the authors' classification 8 of 13 defects would have failed without any user-visible signal, a figure a blind re-rating supported while showing it rests on three borderline judgements. That the discipline's own reference implementation still overstated what it had observed, until review caught it, is the sharpest evidence we can offer that naming the observed event must be a design requirement rather than an aspiration.

Artifact availability. The manuscript, the measurement scripts generating every figure, and the raw observation files are archived on Zenodo under the concept DOI `10.5281/zenodo.23601398`, which resolves to the latest version.

Acknowledgement. We thank the anonymous reviewers, whose comments materially changed Sections 3.2, 3.3 and 5.6.

References

1. United Nations Population Fund: India Ageing Report 2023: Caring for Our Elders — Institutional Responses. UNFPA India, New Delhi (2023)
2. Data for India: Access to phones and the internet (2025)
3. Chakraborty, D., Garg, C.: Navigating technology: mobile media usage and reticence among older adults in rural India. Mobile Media & Communication **14**, 31–49 (2025)
4. Khare, A., Mukherjee, R., Bedarkar, M.: Beyond digital literacy: a structural model analysis of technology acceptance among elderly persons. Humanities and Social Sciences Communications **13**, 794 (2026)
5. Poorcheraghi, H., Negarandeh, R., Pashaeypoor, S., Jorian, J.: Effect of using a mobile drug management application on medication adherence and hospital readmission among elderly patients with polypharmacy: a randomized controlled trial. BMC Health Services Research **23**, 1192 (2023)

6. Salmensuu, O., Isotalo, J., Rijken, M., Hyttinen-Huotari, V., Kaarakainen, M., Linnosmaa, I.: Effects of using medication reminder technologies by home-dwelling older citizens: a systematic review. Age and Ageing **55**(2), afag007 (2026)
7. Awan, M., Ali, S., Ali, M., Abrar, M.F., Ullah, H., Khan, D.: Usability barriers for elderly users in smartphone app usage: an analytical hierarchical process-based prioritization. Scientific Programming **2021**, 2780257 (2021)
8. Elguera Paez, L., Zapata Del Río, C.: Elderly users and their main challenges usability with mobile applications: a systematic review. In: Design, User Experience, and Usability. LNCS, pp. 423–438. Springer, Cham (2019)
9. Liu, N., Yin, J., Tan, S.S.-L., Ngiam, K.Y., Teo, H.H.: Mobile health applications for older adults: a systematic review of interface and persuasive feature design. Journal of the American Medical Informatics Association **28**, 2483–2501 (2021)
10. Gomez-Hernandez, M., Ferre, X., Moral, C., Villalba-Mora, E.: Design guidelines of mobile apps for older adults: systematic review and thematic analysis. JMIR mHealth and uHealth **11**, e43186 (2023)
11. Amouzadeh, E., Dianat, I., Faradmal, J., Babamiri, M.: Optimizing mobile app design for older adults: systematic review of age-friendly design. Aging Clinical and Experimental Research **37**, 248 (2025)
12. Quinn, C.C., Staub, S., Barr, E., Gruber-Baldini, A.: Mobile support for older adults and their caregivers: dyad usability study. JMIR Aging **2**(1), e12276 (2019)
13. Benmessaoud, C., Pfisterer, K.J., De Leon, A., Saragadam, A., El-Dassouki, N., Young, K.G.M., Lohani, R., Xiong, T.: Design of a dyadic digital health module for chronic disease shared care: development study. JMIR Human Factors **10**, e45035 (2023)
14. Android Developers: Optimize for Doze and App Standby (2024)
15. Wiecek, E., Taylor, S., Rourke, H., Hammond, F., Amador-Fernandez, N.: Mobile health apps for older adults: real-world evidence on engagement and medication adherence. Frontiers in Digital Health **8**, 1716880 (2026)
16. Nimmanterdwong, Z., Boonviriya, S., Tangkijvanich, P.: Human-centered design of mobile health apps for older adults: systematic review and narrative synthesis. JMIR mHealth and uHealth **10**(1), e29512 (2022)
17. Hamill, M., Young, V., Boger, J., Mihailidis, A.: Development of an automated speech recognition interface for personal emergency response systems. Journal of NeuroEngineering and Rehabilitation **6**, 26 (2009)
18. Google Play Console Help: Use of SMS or Call Log permission groups (2024)
19. Android Developers Blog: Reminder: SMS/Call Log policy changes (2019). Android Developers Blog (2019)
20. Material Design 3: Accessibility and touch target size (2024)
21. W3C: Web Content Accessibility Guidelines (WCAG) 2.2. W3C Recommendation (2023)